

# CSCI 232

## 232 Algorithms & Data Structures

### LECTURE: 7

J. L. Pach

# OUTLINE:

- Stack
- Queue
- Structs Are Value Types
- Structs as Function Arguments
- Naming Convention Ptr Suffix
- The Double Pointer Rule
- Core operations

# STACK

**(LIFO – Last In, First Out)**

# STACK (LIFO – LAST IN, FIRST OUT)

- Last element pushed is the first to be popped.
- Operations:
  - push(x) → add to the top
  - pop() → remove from the top

```
Stack (LIFO):

  +-----+
  |  D  | <- Top (newest element)
  +-----+
  |  C  |
  +-----+
  |  B  |
  +-----+
  |  A  | <- Bottom (oldest element)
  +-----+

push(E) => E goes on top
pop()   => removes D from the top
```

# Stack — Example




> The element added **last** is removed **first**.

# QUEUE

**(FIFO – First In, First Out)**

# QUEUE (FIFO – FIRST IN, FIRST OUT)

- First element enqueued is the first to be dequeued.
- Operations:
  - enqueue(x) → add to the rear
  - dequeue() → remove from the front

```
Queue (FIFO):

Front -> +-----+     +-----+     +-----+     +-----+ <- Rear
         | A | --> | B | --> | C | --> | D |
         +-----+     +-----+     +-----+     +-----+

enqueue(E) => E goes to the rear
dequeue()  => removes A from the front
```

# Queue — Example

```
enqueue(E):

Front -> [ A ] -> [ B ] -> [ C ] -> [ D ] -> [ E ] <- Rear

dequeue():

        [ A ] removed
Front -> [ B ] -> [ C ] -> [ D ] -> [ E ] <- Rear
```

> The element added **first** is removed **first**.

# STACK & QUEUE

The difference between a stack and a queue is not in the underlying structure—they both work on linked lists—but in the way elements are added and removed.

- In a stack, elements are inserted at the front and also removed from the front (LIFO).
- In a queue, elements are inserted at the rear but removed from the front (FIFO).

| Structure | Insert | Remove | Rule |
|---|---|---|---|
| **Stack** | Top | Top | Last In, First Out |
| **Queue** | Rear | Front | First In, First Out |

# STRUCTS ARE

## Value Types

# STRUCTS ARE VALUE TYPES

In C, a struct is a **value type** — just like `int` or `double`. When you assign a struct to another variable, the **entire content is copied**. Both variables are independent — changing one does not affect the other.

```c
typedef struct {
    int value;
} Node;
Node a;
a.value = 10;
Node b = a;       /* full copy — b gets its own value field */
b.value = 99;
printf("%d\n", a.value);   // 10 — unchanged
printf("%d\n", b.value);   // 99
```

> `b` is a completely independent copy of `a`. There is no shared memory between them.

# STRUCTS AS FUNCTION ARGUMENTS 1/2

> The same rule applies when passing a struct to a function. The function receives a **full copy** — it cannot modify the original.

```c
void setZero(Node n)
{
    n.value = 0;    // modifies the LOCAL copy only
}
int main(void)
{
    Node nodeA;
    nodeA.value = 42;
    setZero(nodeA);
    printf("%d\n", nodeA.value);   // still 42 — original unchanged
}
```



# STRUCTS AS FUNCTION ARGUMENTS 2/2

```c
void setZero(Node n)
{
    n.value = 0;
    // modifies the LOCAL copy only
}
int main(void)
{
    Node nodeA;
    nodeA.value = 42;
    setZero(nodeA);
    printf("%d\n", nodeA.value);
    // still 42 — original unchanged
}
```



```c
void setZero(Node *nodePtr)
{
    nodePtr->value = 0;   // modifies the original
}
```



> To modify a struct inside a function — pass a **pointer** to it.

# NAMING CONVENTION

**Ptr Suffix**

# NAMING CONVENTION — `PTR` SUFFIX

Every variable that is a pointer gets the suffix `Ptr` in its name.

This makes the type visible directly in the name — no need to check the declaration.

```
Node  *headPtr       // pointer to Node
Node **headPtr2Ptr   // pointer to pointer to Node
Node  *currentPtr    // pointer used for traversal
Node  *newNodePtr    // pointer to a new node
Node  *tempPtr       // temporary pointer
```



> If you see `Ptr` at the end — it is a pointer.
> If you see `Ptr2Ptr` at the end — it is a pointer to a pointer.
> No guessing. No checking declarations.

# WHY THIS MATTERS AT THE CALL SITE

```
// headPtr  → pass directly
printList(headPtr);

// headPtr2Ptr → pass address of pointer
addFirst(&headPtr, &nodes[0]);
```



When you call a function and see `&headPtr` — you immediately know:

the function receives a `headPtr2Ptr` and **can modify** your `headPtr`.

When you call a function and see just `headPtr` — the function

reads the list but **cannot change** where `headPtr` points.

# NODE STRUCTURE

## & typedef

# STRUCT WITHOUT TYPEDEF

```c
struct Node
{
    int value;
    struct Node *next;   // must write "struct Node" — typedef not yet complete
};

struct Node  nodeA;              // must write "struct Node" everywhere
struct Node *headPtr = NULL;
```



> Every declaration requires the full `struct Node`.
> Verbose but explicit — you always know it is a struct.

# STRUCT WITH TYPEDEF

```c
typedef struct Node
{
    int value;
    struct Node *next;   // still "struct Node" here — typedef not complete yet
} Node;

Node  nodeA;            // now "Node" alone is enough
Node *headPtr = NULL;
```



`typedef` creates an alias.

Inside the struct body the alias is not yet defined — `struct Node *next` is still required there.

Everywhere else — `Node` alone works.

# STRUCT WITH TYPEDEF + FORWARD DECLARATION

```c
typedef struct Node Node;   // tell the compiler "Node" will exist

struct Node
{
    int value;
    Node *next;             // alias already known — works here too
};

Node  nodeA;
Node *headPtr = NULL;
```



Cleanest form.

Forward declaration lets you use `Node` even inside the struct body.

We use this form for the rest of the course.

# STACK MEMORY

**No malloc**

# STACK MEMORY — NO MALLOC

Memory comes from two places:

- **Stack** — local variables, managed automatically by the compiler
- **Heap** — `malloc()` / `free()`, managed manually by the programmer

**In this lecture we work exclusively on the stack.**

No `malloc()`. No `free()`. No memory leaks.

The pointer logic is **identical** to heap-based lists.

Once this is solid, switching to `malloc()` is a small step.

# Nodes Live in an Array in main()

```c
int main(void)
{
    Node  nodes[4];       /* all nodes live here — on the stack */
    Node *headPtr = NULL;  /* headPtr starts empty */
    for (int i = 0; i < 4; i++)
    {
        nodes[i].value  = (i + 1) * 10;
        nodes[i].next   = NULL;
    }
    /* pass addresses of stack nodes to list functions */
    addFirst(&headPtr, &nodes[0]);   // headPtr → nodes[0]
    addFirst(&headPtr, &nodes[1]);   // headPtr → nodes[1] → nodes[0]

    printList(headPtr);
    return 0;
}
```

> Nodes live in `nodes[]` for the entire lifetime of `main()`.

# WHY NOT CREATE NODES INSIDE FUNCTIONS?

```c
Node *createNode_BROKEN(int value) /* DANGEROUS — do not do this */
{
    Node newNode;          /* lives only inside this function */
    newNode.value = value;
    newNode.next  = NULL;
    return &newNode;       /* dangling pointer — node is gone after return */
}
void initNode(Node *nodePtr, int value) /* SAFE — node lives in main(), function only initializes it */
{
    nodePtr->value = value;
    nodePtr->next  = NULL;
}
```



> A function can **modify** a node through a pointer.
>
> A function cannot **create** a node on the stack and return its address.

# THE DOUBLE

## Pointer Rule

# THE PROBLEM — SINGLE POINTER

```c
void addFirst_BROKEN(Node *headPtr, Node *newNodePtr)
{
    newNodePtr->next = headPtr;
    headPtr = newNodePtr;   // changes the LOCAL copy only
}                           // caller's headPtr is unchanged
int main(void)
{
    Node  nodes[1];
    Node *headPtr = NULL;
    initNode(&nodes[0], 10);
    addFirst_BROKEN(headPtr, &nodes[0]);
    // headPtr is still NULL here
}
```



> The function received a **copy** of `headPtr`. Changing the copy does not change the original.

# THE RULE 1/2

**Every function that modifies the list receives `Node **headPtr2Ptr`.**

Pass the address of `headPtr` using `&`:

```
addFirst(&headPtr, &nodes[0]);   // &headPtr gives the function direct access
```



Inside the function:

- `headPtr2Ptr` holds the **address** of `headPtr` in `main()`
- `*headPtr2Ptr` IS `headPtr` in `main()` — not a copy
- `(*headPtr2Ptr)->value` accesses the first node's field

# THE RULE 2/2

Read-only functions receive `Node *headPtr` — no modification needed.

```
/* modifying — always double pointer */
int addFirst   (Node **headPtr2Ptr, Node *newNodePtr);
int addLast    (Node **headPtr2Ptr, Node *newNodePtr);
int removeFirst(Node **headPtr2Ptr);
int deleteValue(Node **headPtr2Ptr, int value);

/* read-only — single pointer is sufficient */
int printList  (Node  *headPtr);
```



# DOUBLE POINTER — MEMORY PICTURE

```
main():

    headPtr  [ ●──────────────────────────►  [ 10 | NULL ] ]
    &headPtr = address of headPtr variable

Inside addFirst(Node **headPtr2Ptr, Node *newNodePtr):

    headPtr2Ptr  ─────►  headPtr in main()
    *headPtr2Ptr  =    headPtr in main()    ← we write here directly

After *headPtr2Ptr = newNodePtr:

    headPtr  [ ●──►  [ 20 | ●──►  [ 10 | NULL ] ] ]
```

> `*headPtr2Ptr = newNodePtr` changes `main()`'s `headPtr` directly.
>
> No copy. No return value needed.

# CORE

## Operations

# INITNODE

```c
void initNode(Node *nodePtr, int value)
{
    nodePtr->value = value;
    nodePtr->next  = NULL;
}
```



- Receives a pointer to an **existing** node — allocated by the caller.
- Sets `value` and clears `next`.
- Does not allocate memory — the caller owns the node.

```c
Node nodes[3];
initNode(&nodes[0], 10);
initNode(&nodes[1], 20);
initNode(&nodes[2], 30);
```



# PRINTLIST 1/2

```c
int printList(Node *headPtr)
{
    if (headPtr == NULL)
    {
        printf("List is empty.\n");
        return -1;
    }
    Node *currentPtr = headPtr;
    while (currentPtr != NULL)
    {
        printf("%d\n", currentPtr->value);
        currentPtr = currentPtr->next;
    }
    return 0;
}
```



# PRINTLIST 2/2

- Read-only → single pointer is sufficient.
- `currentPtr` is a local traversal variable — moving it does not affect the caller's `headPtr`.
- Loop ends when `currentPtr` reaches `NULL`.

# ADDFIRST — BEFORE & AFTER

```
BEFORE addFirst(&headPtr, &nodes[2]):

    headPtr ──► [ 10 | ──► [ 20 | NULL ] ]

    newNodePtr ──► [ 30 | NULL ]

AFTER:

    headPtr ──► [ 30 | ──► [ 10 | ──► [ 20 | NULL ] ] ]
```

New node is placed at the **front**.

`headPtr` now points to the new node.

Works for both empty and non-empty lists.

# ADDFIRST

```c
int addFirst(Node **headPtr2Ptr, Node *newNodePtr)
{
    if (headPtr2Ptr == NULL)
    {
        printf("Error: headPtr2Ptr is NULL.\n");
        return -1;
    }
    if (newNodePtr == NULL)
    {
        printf("Error: newNodePtr is NULL.\n");
        return -1;
    }
    newNodePtr->next = *headPtr2Ptr;   // new node points to current first
    *headPtr2Ptr     = newNodePtr;    // headPtr now points to new node
    return 0;
}
```



- `newNodePtr->next = *headPtr2Ptr` works even when the list is empty —

# ADDLAST — BEFORE & AFTER

```
BEFORE addLast(&headPtr, &nodes[2]):

    headPtr ──► [ 10 | ──► [ 20 | NULL ] ]

    newNodePtr ──► [ 30 | NULL ]

AFTER:

    headPtr ──► [ 10 | ──► [ 20 | ──► [ 30 | NULL ] ] ]
```

New node is placed at the **rear**.

`headPtr` does not change — unless the list was empty.

Walk to the last node, then attach.

# ADDLAST

```
// ha ha ;)
```



# End-to-End Example



```
int main(void)
{
    Node  nodes[4];
    Node *headPtr = NULL;
    initNode(&nodes[0], 10);
    initNode(&nodes[1], 20);
    initNode(&nodes[2], 30);
    initNode(&nodes[3], 40);
    addLast(&headPtr, &nodes[0]);    // 10
    addLast(&headPtr, &nodes[1]);    // 10 -> 20
    addLast(&headPtr, &nodes[2]);    // 10 -> 20 -> 30
    addFirst(&headPtr, &nodes[3]);   // 40 -> 10 -> 20 -> 30
    printList(headPtr);
    // 40
    // 10
    // 20
    // 30
    return 0;
}
```

# BEST PRACTICES

# HEAD AND TAIL ARE JUST REFERENCES

`headPtr`, `tailPtr`, `currentPtr` — these are all just pointers.

They point to nodes, but they **do not own** them.

Nodes live in the `nodes[]` array in `main()`.

```
Node *headPtr = &nodes[0];
Node *tailPtr  = &nodes[2];

// headPtr and tailPtr point to the same memory as nodes[0] and nodes[2]
// there is no duplication — just two pointers to existing nodes
```

> Changing `headPtr = headPtr->next` does not destroy any node.
> It only changes what `headPtr` points to.
> The node in `nodes[0]` still exists unchanged.

# USE PARENTHESES FOR CLARITY

When chaining pointer operations, add parentheses to make the intent explicit. This is especially helpful while you are learning pointer traversal.

```
// harder to follow — which -> binds first?
current->next->next
// intent is clear — step by step
(current->next)->next
```



```
// harder to follow
(*headPtr2Ptr)->value
// reads naturally — dereference first, then access field
(*headPtr2Ptr)->value   // already clear with the parentheses
```



> Once pointer traversal becomes intuitive, parentheses are optional. Until then — use them.

# NEVER DEREFERENCE NULL

```c
// WRONG — crash if headPtr is NULL
printf("%d\n", headPtr->value);

// CORRECT — guard before every dereference
if (headPtr != NULL)
{
    printf("%d\n", headPtr->value);
}
```



> Every pointer access must be guarded.
>
> `headPtr == NULL` means the list is empty — that is valid state, not a bug.
>
> Dereferencing `NULL` is always a bug.

# COPYING A POINTER COPIES THE ADDRESS

```
Node *aPtr = headPtr;
Node *bPtr = aPtr;     // both point to the same node
```



```
headPtr ──►
aPtr    ──► [ 10 | ──► [ 20 | NULL ] ]
bPtr    ──►
```

Three pointers, one node in memory.

Changing `bPtr->value` changes what `aPtr` and `headPtr` also see.

There is no copy of the data — only a copy of the address.

# DRAW BEFORE YOU CODE

Before writing any function that modifies a list — draw the state **before** and **after** on paper.

```
addFirst — before:
    headPtr ──► [ 10 | ──► [ 20 | NULL ] ]
    newNodePtr ──► [ 5 | NULL ]

addFirst — after:
    headPtr ──► [ 5 | ──► [ 10 | ──► [ 20 | NULL ] ] ]
```

> If you cannot draw it — you cannot code it.
> Paper is faster than the debugger at this stage.

# Summary

| Rule | Why |
|---|---|
| `Ptr` suffix for every pointer | type visible in the name |
| `headPtr2Ptr` for modifying functions | makes `**` explicit at declaration |
| Single `headPtr` for read-only functions | no `**` needed when not modifying |
| Nodes live in `main()` | no dangling pointers, no malloc |
| Guard every dereference | `NULL` is valid state, not a bug |
| Draw before you code | pointer logic is spatial — visualize it |

# THANK

You